

Study Note: Solving the 2D Ackley Function with a Genetic Algorithm

1. The Problem

Goal: find the values of x_1, x_2 that give the smallest possible output of the 2D Ackley function.

$$f(x_1, x_2) = -a \cdot \exp(-b \cdot \sqrt{0.5 \cdot (x_1^2 + x_2^2)}) - \exp(0.5 \cdot (\cos(c \cdot x_1) + \cos(c \cdot x_2))) + a + \exp(1)$$

with $a = 20$, $b = 0.2$, $c = 2\pi$, searched over $-5 \leq x_1, x_2 \leq 5$.

Why it's hard: the `cos` terms create *thousands of small bumpy local minima* across the search space (look at any plot of it — it's like an egg carton). A simple hill-descending method can easily get trapped in one of those bumps instead of reaching the true lowest point. The **known answer** is $f(0, 0) = 0$ — a single global minimum right at the origin.

This is a textbook benchmark used to prove an optimizer can escape local traps, not just walk downhill.

2. Why a Genetic Algorithm (GA)?

A GA doesn't rely on gradients or a single starting point. It keeps a whole *population* of candidate solutions, and evolves them generation by generation using ideas borrowed from natural selection: the fitter solutions are more likely to reproduce, and small random changes (mutation) let the population explore new areas — which is exactly what you need to jump out of Ackley's many local bumps.

3. Our Solution — Setup

Element	Choice	Why
Chromosome	<code>[x1, x2]</code> — real numbers (value encoding), not binary	The problem is naturally 2 real-valued variables
Population size	50	Enough diversity to cover the search space
Generations	100 (stop early if $\text{fitness} \leq 1e-6$)	Gives the population time to converge
Fitness function	The Ackley function itself	We are <i>minimizing</i> it — lower = better

Element	Choice	Why
Selection	Roulette Wheel	Fitter individuals get a bigger "slice" of the wheel; since we minimize, fitness is inverted first (<code>worst - f</code>) so low-Ackley individuals get the biggest slice
Crossover	1-point, at 80% probability	With only 2 genes, the one possible cut swaps <code>x2</code> between two parents: <code>[p1.x1, p2.x2]</code> and <code>[p2.x1, p1.x2]</code>
Mutation	1-point gene mutation, at 5% probability	Pick one gene at random, nudge it with a small Gaussian random value, clip back into <code>[-5, 5]</code>
Elitism	Top 1 individual	The single best solution is always carried into the next generation untouched, so progress never goes backward

4. The Algorithm, Step by Step

1. **Initialize:** create 50 random chromosomes `[x1, x2]` , each gene uniform in `[-5, 5]` .
2. **Evaluate:** compute `f(x1, x2)` for every chromosome (its fitness).
3. **Sort & Elitism:** sort the population best→worst; set aside the top 1 unchanged.
4. **Selection:** use Roulette Wheel to repeatedly pick 2 parents (fitter = more likely).
5. **Crossover:** with 80% probability, swap `x2` between the two parents to create 2 children.
6. **Mutation:** with 5% probability, perturb one gene of a child by a small random amount.
7. **Replace:** next generation = elite (1) + newly bred children (49) = 50 again.
8. **Repeat** steps 2–7 until 100 generations are done, or the best fitness gets $\leq 1e-6$ (i.e., essentially 0).

5. Results

Metric	Value
Best solution found	<code>x1 = -0.0018 , x2 = 0.0050</code>
Best fitness <code>f(x1, x2)</code>	0.0157 (true minimum is exactly 0)
Generations used	100 (didn't need early stopping, but got very close very fast)
Got within ~0.07 of the minimum by generation	~20

Convergence behavior: the best-so-far fitness drops sharply for the first ~20 generations (8.0 → ~0.07), then fine-tunes slowly for the rest of the run — classic GA behavior: fast global search early, then local refinement.

Comparison with a classical optimizer (Nelder-Mead, started at $(4, -4)$):

- Classical method got stuck at $f \approx 11.0$ — trapped in a local bump near its starting point.
- The GA reached $f \approx 0.016$ — because it searches many regions in parallel instead of committing to one starting point.
- **Takeaway:** this is *the* key selling point of GAs on Ackley — classical local search gets trapped, population-based search does not.

Reduced-generation runs (to show how quality scales):

- 20 generations → $f \approx 0.066$
- 100 generations → $f \approx 0.016$
- More generations → more fine-tuning, though most of the "hard work" (escaping local bumps) already happens in the first ~20 generations.

6. Talking Points for Presenting

- **"Why GA and not gradient descent?"** → Ackley has too many local minima; GA's population + mutation lets it escape them, gradient descent from a single point usually can't.
- **"Why invert fitness for selection?"** → Roulette Wheel naturally favors *bigger* numbers, but we want to favor *smaller* Ackley values, so we flip it ($\text{worst} - f$) before computing selection probabilities.
- **"Why does elitism matter?"** → Without it, a lucky best solution could be lost to random crossover/mutation; with it, the best-ever answer is guaranteed to survive.
- **"How close did we get?"** → 0.0157 vs. the true 0 , i.e., within about 1.6% of the answer scale — essentially solved, visually indistinguishable from the true minimum on the Ackley surface plot.

7. Where to Find the Evidence (for the demo)

- `src/output/convergence_curve.png` — best & mean fitness dropping toward 0 across generations.
- `src/output/ackley_surface_solution.png` — GA's final answer plotted right on top of the true minimum.
- `src/output/detailed/evolution.gif` — animation of the population converging, generation by generation.
- `src/output/detailed/GA_Detailed_Report.xlsx` — full per-generation tables if asked to show the raw data.